

11. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_CMD 100

int main() {
    char history[MAX_HISTORY][MAX_CMD];
    char command[MAX_CMD];
    int count = 0;

    printf("Simple Command History\n");
    printf("Type 'history' to display commands\n");
    printf("Type 'exit' to quit\n\n");

    while (1) {
        printf("shell> ");
        fgets(command, MAX_CMD, stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0) {
            printf("\nCommand History:\n");

            for (int i = 0; i < count; i++) {
                printf("%d %s\n", i + 1, history[i]);
            }

            continue;
        }

        if (strlen(command) == 0)
            continue;

        if (count < MAX_HISTORY) {
            strcpy(history[count], command);
            count++;
        }
    }
}
```

```

        continue;
    }

    if (strlen(command) == 0)
        continue;

    if (count < MAX_HISTORY) {
        strcpy(history[count], command);
        count++;
    } else {
        for (int i = 0; i < MAX_HISTORY - 1; i++)
            strcpy(history[i], history[i + 1]);

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

return 0;
}

```

OUTPUT:

```

hp@Hima:~$ nano skill11.c
hp@Hima:~$ gcc skill11.c -o skill11
hp@Hima:~$ ./skill11
Simple Command History
Type 'history' to display commands
Type 'exit' to quit

shell> ls
shell> pwd
shell> date
shell> history

Command History:
1  ls
2  pwd
3  date
shell> |

```

12. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd[2];
    pid_t p1, p2;

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    p1 = fork();

    if (p1 == 0) {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);

        execlp("ls", "ls", NULL);

        perror("execlp");
        exit(1);
    }

    p2 = fork();

    if (p2 == 0) {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);

        execlp("wc", "wc", "-l", NULL);
```

```

        execlp("wc", "wc", "-l", NULL);

        perror("execlp");
        exit(1);
    }

    close(fd[0]);
    close(fd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    return 0;
}

```

OUTPUT:

```

hp@Hima:~$ $ nano skill12.c
hp@Hima:~$ $ gcc skill12.c -o skill12
hp@Hima:~$ $ ./skill12
60

```

13.To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        int fd = open("input.txt", O_RDONLY);

        if (fd < 0) {
            perror("open");
            exit(1);
        }

        dup2(fd, STDIN_FILENO);
        close(fd);

        execlp("wc", "wc", "-l", NULL);

        perror("execlp");
        exit(1);
    }

    wait(NULL);

    return 0;
}

```

```

hp@Hima:~$ nano skill13.c
hp@Hima:~$ gcc skill13.c -o skill13
hp@Hima:~$ ./skill13
open: No such file or directory

```

14. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        int out = open("output.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

        int err = open("error.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

        if (out < 0 || err < 0) {
            perror("open");
            exit(1);
        }

        dup2(out, STDOUT_FILENO);
        dup2(err, STDERR_FILENO);

        close(out);
        close(err);

        execlp("ls", "ls", "nonexistent_file", NULL);

        perror("ls");
        exit(1);
    }

    wait(NULL);

```

```

    wait(NULL);

    printf("Output written to output.txt\n");
    printf("Error written to error.txt\n");

    return 0;
}

```

OUTPUT:

```
hp@Hima:~$ nano skill14.c
hp@Hima:~$ gcc skill14.c -o skill14
hp@Hima:~$ ./skill14
Output written to output.txt
Error written to error.txt
hp@Hima:~$ |
```

15. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pipefd[2];

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 == 0) {
        int fd = open("result.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

        if (fd < 0) {
            perror("open");
            exit(1);
        }

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        execlp("ls", "ls", NULL);

        perror("ls");
        exit(1);
    }
}

```

```

pid_t p2 = fork();

if (p2 == 0) {
    int fd = open("result.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    if (fd < 0) {
        perror("open");
        exit(1);
    }

    dup2(pipefd[0], STDIN_FILENO);
    dup2(fd, STDOUT_FILENO);

    close(fd);
    close(pipefd[0]);
    close(pipefd[1]);

    execlp("wc", "wc", "-l", NULL);

    perror("wc");
    exit(1);
}

close(pipefd[0]);
close(pipefd[1]);

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

printf("Result stored in result.txt\n");

return 0;
}

```

OUTPUT:

```

hp@Hima:~$ nano skill15.c
hp@Hima:~$ gcc skill15.c -o skill15
hp@Hima:~$ ./skill15
Result stored in result.txt
hp@Hima:~$ |

```

16. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.To

Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Background process started. PID = %d\n", getpid());

        sleep(5);

        printf("Background process completed.\n");

        exit(0);
    }

    printf("Parent continues immediately.\n");
    printf("Background PID = %d\n", pid);

    printf("Parent is not waiting immediately.\n");

    sleep(2);

    waitpid(pid, NULL, 0);

    printf("Background job finished.\n");

    return 0;
}
```

OUTPUT:

```
hp@Hima:~$ nano skill16.c
hp@Hima:~$ gcc skill16.c -o skill16
hp@Hima:~$ ./skill16
Parent continues immediately.
Background PID = 1356
Parent is not waiting immediately.
Background process started. PID = 1356
Background process completed.
Background job finished.
hp@Hima:~$ !
```

